Applications Documentation

SaltOS 4.1 r2757

October 2026

Contents

1 Certs **10**

1.1 Certificate Management Functions . . . 10

1.1.1 List Certificates . . . 10

1.1.2 Insert Certificates . . . 10

1.1.3 Convert Certificate Hash to Nickname . . . 10

1.1.4 Check Certificate Existence . . . 10

1.1.5 View Certificate Information . . . 10

1.1.6 Delete Certificate . . . 11

1.2 Directory helper . . . 11

1.2.1 Passthru helper . . . 11

1.2.2 Init nssdb repo . . . 11

1.2.3 Create certificate . . . 11

1.2.4 Add certificate . . . 11

1.2.5 List certificates . . . 12

1.2.6 Info of a certificate . . . 12

1.2.7 Pdf signature . . . 12

1.2.8 Remove certificate . . . 12

1.2.9 Reset nssdb repo . . . 12

1.2.10 FPDI and FPDF Extensions Bootstrapper . . . 12

1.2.11 FPDI Canvas Extended with Text Rotation Capabilities . . . 12

1.2.12 Update pdf . . . 13

2 Common **13**

2.1 Profile application . . . 13

2.1.1 Main object . . . 13

2.1.2 Initialization of profile settings . . . 13

2.1.3 Update authentication settings . . . 13

2.2 Data Actions Merger . . . 13

2.2.1 Merge data with actions . . . 14

2.3 DB stats management functions . . . 14

2.3.1 Get the DB stats connection . . . 14

2.3.2 List DB stats . . . 14

2.3.3 Format a DB stats row . . . 14

2.3.4 Check DB stats row existence . . . 15

2.3.5 View DB stats row . . . 15

2.4 Apps helper module . . . 15

2.4.1 Make app file . . . 15
2.5 File management functions . . . 16
2.5.1 List files in the logs directory . . . 16
2.5.2 Get the full path of a file . . . 16
2.5.3 Check if a file exists . . . 16
2.5.4 View the contents of a file . . . 16
2.6 Help feature . . . 17
2.6.1 Detect Help File . . . 17
2.7 Matrix functions . . . 17
2.7.1 Create matrix for version data . . . 17
2.7.2 Create matrix for log data . . . 17
2.8 Helper file for the order feature . . . 17
2.8.1 Get Order Fields . . . 17
2.9 PDF helper file . . . 17
2.9.1 Build the expected PDF template file path for the given app. . . . 18
2.9.2 Check if the PDF layout file exists for the given app. . . . 18

3 Crm **18**
3.1 Quotes application . . . 18
3.1.1 Main object . . . 18
3.1.2 Fix tax dropdown visibility. . . . 18
3.1.3 Lines columns . . . 18
3.1.4 Types . . . 19
3.1.5 Handle and recalculate quote line totals and tax breakdowns after user edits. . . . 19
3.1.6 Apply the tax or the withholding of the document to all lines . . . 19
3.2 Reverses the process of ‘make_matrix_data()‘ for the ‘quotes‘ app. . . . 20
3.2.1 unmake_matrix_data . . . 20
3.2.2 Get next code . . . 20
3.2.3 Generates or completes quote and final quote metadata. . . . 20

4 Dashboard **20**
4.1 Dashboard application . . . 20
4.1.1 Initialization of dashboard . . . 21
4.2 Dashboard and navbar generation logic. . . . 21
4.2.1 Build the default dashboard layout with groups and application buttons. . . . 21
4.2.2 Build the application menu for the navbar. . . . 21

5 Emails **22**
5.1 Email application . . . 22

5.1.1 Driver type2emails object . . . 22
5.1.2 Create type2emails template . . . 22
5.1.3 Initialization, open, and close handlers for type2emails . . . 22
5.2 Email application . . . 22
5.2.1 Main object . . . 22
5.2.2 Initialize the email application . . . 22
5.2.3 Perform server-related actions for emails . . . 23
5.2.4 Delete selected emails . . . 23
5.2.5 Delete a single email . . . 23
5.2.6 Send an email . . . 23
5.2.7 Set email state . . . 23
5.2.8 Add signature to email . . . 23
5.2.9 View email source . . . 23
5.2.10 Reply to an email . . . 23
5.2.11 Reply to all recipients of an email . . . 24
5.2.12 Forward an email . . . 24
5.2.13 Driver search . . . 24
5.3 Email Query Builder . . . 24
5.3.1 Generate WHERE query for emails . . . 24
5.4 Get email library . . . 24
5.4.1 Requires section . . . 24
5.4.2 Defines section . . . 25
5.4.3 Remove all body . . . 25
5.4.4 Process message . . . 25
5.4.5 Process plain html . . . 25
5.4.6 Process file . . . 25
5.4.7 Check permissions . . . 25
5.4.8 Get GZ File . . . 25
5.4.9 Get source . . . 26
5.4.10 Mime decode protected . . . 26
5.4.11 Get mime . . . 26
5.4.12 Get Node . . . 26
5.4.13 Get type . . . 26
5.4.14 Get disposition . . . 26
5.4.15 Get files . . . 27
5.4.16 Get info . . . 27
5.4.17 Fix string . . . 27

5.4.18 Get text body . . . 27
5.4.19 Get full body . . . 27
5.4.20 Get cid . . . 27
5.4.21 Insert . . . 28
5.4.22 Update . . . 28
5.4.23 Add bcc . . . 28
5.4.24 Gzfile size . . . 28
5.4.25 Get email body . . . 28
5.4.26 Generate formatted email header information . . . 29
5.4.27 Extract and format the body of an email . . . 29
5.4.28 Get email source . . . 29
5.4.29 Get email files . . . 29
5.4.30 Get cid . . . 29
5.4.31 Is outbox . . . 29
5.4.32 Server . . . 30
5.4.33 Delete . . . 30
5.4.34 Get viewpdf . . . 30
5.4.35 Get download . . . 30
5.4.36 Update email states . . . 30
5.4.37 Generate PDF from emails . . . 30
5.4.38 Generate iframe content with secure policies . . . 31
5.5 Html helper module . . . 31
5.5.1 Defines section . . . 31
5.5.2 Remove Script Tag . . . 31
5.5.3 Remove Style Tag . . . 31
5.5.4 Remove Comment Tag . . . 31
5.5.5 Remove Meta Tag . . . 32
5.5.6 Remove Link Tag . . . 32
5.5.7 Inline Img Tag . . . 32
5.5.8 Inline Img Style . . . 32
5.5.9 Inline Img Background . . . 32
5.5.10 Inline Image Helper . . . 32
5.5.11 Extract Inline Images from Tags . . . 32
5.5.12 Extract Inline Images from Styles . . . 33
5.5.13 Extract Inline Images from Backgrounds . . . 33
5.5.14 Fix Image Tags . . . 33
5.5.15 Fix Image Styles . . . 33

5.5.16 Fix Image Backgrounds . . . 33
5.5.17 Fix Base64-Encoded Files . . . 34
5.6 Make indexing action . . . 34
5.7 Mailparse helper module . . . 34
5.7.1 Internal handle used by the functions below to keep the parsed . . . 35
5.7.2 Mailparse Index Parts Helper . . . 35
5.7.3 Mailparse Unfold Helper . . . 35
5.7.4 Mailparse Encode Display Helper . . . 35
5.7.5 Mailparse Msg Create Helper . . . 35
5.7.6 Mailparse Msg Parse Helper . . . 36
5.7.7 Mailparse Msg Get Structure Helper . . . 36
5.7.8 Mailparse Msg Get Part Helper . . . 36
5.7.9 Mailparse Msg Get Part Data Helper . . . 36
5.7.10 Mailparse Msg Extract Part Helper . . . 36
5.7.11 Mailparse Msg Free Helper . . . 36
5.7.12 Mailparse Rfc822 Parse Addresses Helper . . . 36
5.7.13 Thin mailparse_* delegates: only defined when the real extension is . . . 37
5.7.14 Mailparse Msg Create . . . 37
5.7.15 Mailparse Msg Parse . . . 37
5.7.16 Mailparse Msg Get Structure . . . 37
5.7.17 Mailparse Msg Get Part . . . 37
5.7.18 Mailparse Msg Get Part Data . . . 37
5.7.19 Mailparse Msg Extract Part . . . 37
5.7.20 Mailparse Msg Free . . . 37
5.7.21 Mailparse Rfc822 Parse Addresses . . . 37
5.8 SaltOS MIME Parser Component . . . 37
5.8.1 - @param array $input Must be an array with 'File' or 'Data' . . . 38
5.8.2 - @param resource $msg mailparse message handle . . . 38
5.9 SaltOS POP3 Client Component . . . 38
5.9.1 Prepare the cURL handle and compute the base URL based on configuration. . . . 38
5.9.2 Retrieve the server's UIDL listing and return a map of index => uidl. . . . 38
5.9.3 Download a full message by server index using "RETR <index>". . . . 39
5.9.4 Issue "DELE <index>" to mark a message for deletion. . . . 39
5.9.5 Release the cURL handle. No explicit POP3 QUIT is required because each . . . 39
5.9.6 Execute a POP3 command using cURL. . . . 39
5.10 Send email library . . . 39
5.10.1 Used libraries . . . 40

5.10.2 Sendmail . . . 40
5.10.3 Debugoutput helper . . . 40
5.10.4 Parser . . . 40
5.10.5 Message Id . . . 40
5.10.6 Eml saver . . . 40
5.10.7 Obj saver . . . 41
5.10.8 Prepare email for sending . . . 41
5.10.9 Perform email sending action . . . 41
5.10.10 Send queued emails from the server . . . 41
5.10.11 Retrieve email attachments . . . 41
5.10.12 Update email signature, CC, and encryption state . . . 42

6 Sales **42**

6.1 Invoices application . . . 42
6.1.1 Main object . . . 42
6.1.2 Fix tax dropdown visibility. . . . 42
6.1.3 Lines columns . . . 42
6.1.4 Types . . . 42
6.1.5 Handle and recalculate invoice line totals and tax breakdowns after user edits. . . . 42
6.1.6 Apply the tax or the withholding of the document to all lines . . . 43
6.2 ChartJS widgets for invoices . . . 43
6.2.1 compute_invoice_total_by_day . . . 44
6.2.2 compute_invoice_avg_by_day . . . 44
6.2.3 compute_invoice_avg_days_to_pay . . . 44
6.2.4 compute_top5_customers_by_total . . . 44
6.2.5 compute_invoice_paid_vs_pending . . . 44
6.2.6 compute_invoice_open_vs_closed . . . 44
6.3 Reverses the process of ‘make_matrix_data()‘ for the ‘invoices‘ app. . . . 45
6.3.1 unmake_matrix_data . . . 45
6.3.2 Get next code . . . 45
6.3.3 Generates or completes proforma and final invoice metadata. . . . 45
6.4 Taxes and withholdings helpers . . . 46
6.4.1 Get taxes modes . . . 46
6.4.2 Get taxes master . . . 46
6.4.3 Get taxes column . . . 46
6.4.4 Get taxes total column . . . 46
6.4.5 Get taxes subtotal column . . . 46
6.4.6 Get taxes rows . . . 47

6.4.7 Get taxes default . . . 47
6.4.8 Make taxes data . . . 47

7 Tester **47**

7.1 Rating widget example . . . 47
7.1.1 Rating constructor helper . . . 47
7.2 Tester application . . . 48
7.2.1 Tester object . . . 48
7.2.2 Campo 8: Alert functionality . . . 48
7.2.3 Campo 9: Modal functionality . . . 48
7.2.4 Campo 10: Offcanvas functionality . . . 48
7.2.5 Campo 11: Toast functionality . . . 49
7.2.6 Campo 22: Open statistics page . . . 49

8 Users **49**

8.1 Login application . . . 49
8.1.1 Main object . . . 49
8.1.2 Authenticate login function . . . 49
8.1.3 Renew function . . . 49
8.2 Days functions . . . 50
8.2.1 Days to bin . . . 50
8.2.2 Bin to days . . . 50
8.2.3 Fix for days . . . 50
8.3 Groups functions . . . 50
8.3.1 Insert group action . . . 50
8.3.2 Update group action . . . 50
8.3.3 Delete group action . . . 51
8.4 Matrix functions . . . 51
8.4.1 Create matrix data . . . 51
8.4.2 Create matrix cells . . . 51
8.4.3 Unmake matrix data . . . 51
8.4.4 Generate matrix permissions . . . 51
8.4.5 Get permissions from database . . . 51
8.4.6 Get applications from database . . . 51
8.4.7 Create matrix columns . . . 52
8.4.8 Create matrix rows . . . 52
8.5 Users functions . . . 52
8.5.1 Insert user action . . . 52

8.5.2 Update user action 52
8.5.3 Delete user action 52

1 Certs

1.1 Certificate Management Functions

```
apps/certs/php/certs.php
```

This file contains the main functions for managing certificates, including listing, inserting, checking, viewing, and deleting certificates from the NSS database.

1.1.1 List Certificates

```
function __certs_list($search, $order, $offset, $limit)
```

This function retrieves a list of certificates from the NSS database, applies search filters, and paginates the results based on offset and limit parameters.

- @search => Search term or query to filter certificates.
- @offset => Offset for pagination.
- @limit => Maximum number of certificates to retrieve.

Return the list of certificates with their ID and name.

1.1.2 Insert Certificates

```
function __certs_insert($json)
```

This function inserts certificates into the NSS database. It validates the uploaded files, processes them, and handles errors if the import is unsuccessful.

- @json => JSON object containing the certificate files and their details.

Return the status and message of the operation.

1.1.3 Convert Certificate Hash to Nickname

```
function __certs_hash2nick($hash)
```

This function maps a certificate hash to its corresponding nickname in the NSS database.

- @hash => MD5 hash of the certificate nickname.

Return the nickname of the certificate, or an empty string if not found.

1.1.4 Check Certificate Existence

```
function __certs_check($hash)
```

This function checks if a certificate with the given hash exists in the NSS database.

- @hash => MD5 hash of the certificate nickname.

Return true if the certificate exists, false otherwise.

1.1.5 View Certificate Information

```
function __certs_view($hash)
```

This function retrieves detailed information about a certificate using its hash.

- @hash => MD5 hash of the certificate nickname.

Return the status, nickname, and detailed information of the certificate.

1.1.6 Delete Certificate

```
function __certs_delete($hash)
```

This function removes a certificate from the NSS database using its hash.

- @hash => MD5 hash of the certificate nickname.

Return the status of the delete operation.

1.2 Directory helper

```
apps/certs/php/nssdb.php
```

This function returns the nssdb directory used by all functions

1.2.1 Passthru helper

```
function __nssdb_passthru_helper($cmd)
```

This function uses the ob_passthru and improve the output

- @cmd => command line to be executed

1.2.2 Init nssdb repo

```
function __nssdb_init()
```

This function tries to initialize the nssdb component with an empty repo

1.2.3 Create certificate

```
function __nssdb_create($outfile, $outpass, $subject = '', $name = '')
```

This function create a dummy self signed certificate intended to be used in test cases

$outfile => the p12 file $outpass => the password used by the p12 file

1.2.4 Add certificate

```
function __nssdb_add($file, $pass)
```

This function adds the p12 file that must contains a valid certificate to the nssdb repo using the pass if needed

- @file => the p12 file
- @pass => the p12 password

1.2.5 List certificates

```
function __nssdb_list()
```

This function returns the list of valid nicks that you can use with pdfsig

1.2.6 Info of a certificate

```
function __nssdb_info($nick, $shortnames = false)
```

This function returns the info of the nick certificate of the nssdb repo

- @nick => the desired nick used to retrieve info

1.2.7 Pdf signature

```
function __nssdb_pdfsig($nick, $input, $output)
```

This function uses the pdfsig to add the signature to the pdf using the nick certificate of the nssdb repo

- @nick => the desired nick used to sign de pdf
- @input => the desired input file that you want to sign
- @output => the signed pdf file

1.2.8 Remove certificate

```
function __nssdb_remove($nick)
```

This function remove the certificate from the nssdb repo indentified by nick

- @nick => the desired nick that you want to remove

1.2.9 Reset nssdb repo

```
function __nssdb_reset()
```

This function removes the nssdb repo, is the inverted of the init function

1.2.10 FPDI and FPDF Extensions Bootstrapper

```
require_once 'apps/certs/lib/fpdf/vendor/autoload.php';
```

Automatically loads the external dependencies managed via Composer, ensuring that the setasign/fpdi and setasign/fpdf core classes are available in memory.

1.2.11 FPDI Canvas Extended with Text Rotation Capabilities

```
class fpdi_with_rotation extends \setasign\Fpdi\Fpdi
```

Implements native PDF coordinate manipulation vectors via template processing primitives, adding arbitrary text angle rotations using canvas transformation matrices.

1.2.12 Update pdf

```
function __nssdb_update($nick, $input)
```

This function creates a new pdf document using input as source and adding to each page a new box with the important information about the certificate used in the signature process

- @nick => the desired nick used to sign de pdf
- @input => the desired input pdf file where you want to add the cert info

2 Common

2.1 Profile application

```
apps/common/js/profile.js
```

This application implements the typical features associated with user profiles, such as managing themes, language settings, and authentication updates.

2.1.1 Main object

```
saltos.profile = {};
```

Contains all the logic and code for the SaltOS framework related to the profile application.

2.1.2 Initialization of profile settings

```
saltos.profile.init = arg
```

This method initializes the profile settings by setting the current Bootstrap theme, custom CSS theme, and language preferences in the respective input fields.

2.1.3 Update authentication settings

```
saltos.profile.authupdate = async ()
```

This method restores the previous state of the application if necessary, validates required fields, and then updates the authentication credentials using the provided old password, new password, and its confirmation.

A successful update replaces the current token with a fresh one, exactly as a new login would, so besides the success message this also triggers the same saltos.app.login event used by the login and renew screens, to refresh the current view with the new session.

Note: the outcome must to be checked using the response status instead of the token presence, because unlike login/renew, here a valid token already exists before calling this function, so it would stay valid (and truthy) even when the update itself fails

Note: the success message uses saltos.app.toast instead of saltos.app.modal, following the same convention used elsewhere in the app (see driver.js) for successful actions; every hashchange event unconditionally closes any open modal (see hash.js), and the saltos.app.login event triggered right below fires exactly that, so a modal here would close itself before ever being seen

2.2 Data Actions Merger

```
apps/common/php/actions.php
```

This file contains functionality for merging action controls with data rows in tables or lists, including permission checks

2.2.1 Merge data with actions

```
function __merge_data_actions($data, $actions)
```

Combines dataset rows with action controls while verifying permissions. Processes each action definition, checks user permissions, and attaches permitted actions to each data row.

- @data => Dataset containing rows to display in table/list
- @actions => Action definitions to merge with each data row

Returns the modified dataset with actions attached to each row

Throws error if actions parameter is not an array

2.3 DB stats management functions

```
apps/common/php/dbstats.php
```

These functions expose the tbl_slowqueries table collected by __db_query_stats() (see php/lib/dbstats.php) as a read-only list/view application. The table lives in its own isolated SQLite file, not in the main application database, so the queries here connect to it directly instead of going through db_query()/get_config('db/obj')

2.3.1 Get the DB stats connection

```
function __dbstats_connect()
```

This function returns the isolated connection to the stats database, or null when the file does not exist yet (debug/slowquerystats never enabled, or nothing recorded)

Returns the database connector object, or null if the stats file does not exist

2.3.2 List DB stats

```
function __dbstats_list($search, $order, $offset, $limit)
```

This function retrieves a list of rows from the tbl_slowqueries table, applies search filters over the source column, and paginates the results based on offset and limit

- @search => Search term or query to filter rows by source.
- @order => Column and direction to order by, already validated by check_order().
- @offset => Offset for pagination.
- @limit => Maximum number of rows to retrieve.

Return the list of rows with their id, source and aggregated timing stats.

2.3.3 Format a DB stats row

```
function __dbstats_format($row)
```

This function is a helper used by __dbstats_list and __dbstats_view to format a raw tbl_slowqueries row into the fields shown by the application, adding the average time

- @row => the raw row, as returned by the database driver.

Return the formatted row.

2.3.4 Check DB stats row existence

```
function __dbstats_check($id)
```

This function checks if a row with the given id exists in the tbl_slowqueries table.

- @id => Id of the row to check.

Return true if the row exists, false otherwise.

2.3.5 View DB stats row

```
function __dbstats_view($id)
```

This function retrieves the detailed information of a single tbl_slowqueries row.

- @id => Id of the row to view.

Return the formatted row, or a JSON error if the row is not found.

2.4 Apps helper module

```
apps/common/php/default.php
```

This file contains functions intended to be used as hepers of other functions, allowing to convert between formats like yaml to xml using a small specification

2.4.1 Make app file

```
function make_app_file($data)
```

This function returns the xml of an app using the follow specs passed in @data:

- @require => this field defines the required php that contains the make_app_file function like this file
- @_template => the xml app file used as template, this file contains the default specification of the app used in the quick apps defined in the yaml
- @screen => the screen used by the template
- @list => the list of fields used in the list screen, this must be a list of arrays with 3 elements: id, type and label, types can be as follow: -text => default field with text -select => this field resolves the text from a table and field, see the @select spec -options => this field resolves the text from a fixed list of options, see the @options spec -boolean => this field uses an icon with V o X in green or red to see the value of the field (generaly 1 or 0) -hastext => this field is similar to boolean but the true or false is detedmined using the contents of the text, no text is false and some text is true
- @form => the list of fields used in the form screen, this must be a list of arrays with 3 elements: id, type and label, types can be as follow:
  - text, date, time, datetime, checkbox, switch => default widget
  - textarea, joditeditor, codemirror => widget with 5em of height
  - select => widget that uses the select spec to resolve the contents
  - options => widget that uses the options spec to resolve the contents
- @select => this spec is intended to defined the selects used in the list and forms, requires an array of 3 elements: id, table and field, if the last field is not specified, saltos tries to resolve it using the manifest information (internally use the table2field feature), a fourth optional element can be used to define the where applied to the rows of the form widget
- @options => this spec is intended to defined the options used in the list and forms, requires a named array that defines the id of the field and each array entry must to be a list of arrays with 2 elements: value and label

- @attr => this part contain an array with extra features added to the forms widgets, the spec requires to use a named array that defines the id of the field and each array entry must to be another pair of key and val with the name of the property and the value of it

2.5 File management functions

```
apps/common/php/files.php
```

These functions handle file operations such as listing files, retrieving file paths, checking for file existence, and viewing file contents. They are primarily designed to interact with the ‘data/logs‘ directory.

2.5.1 List files in the logs directory

```
function __files_list($search, $order, $offset, $limit)
```

This function retrieves a list of files from the ‘data/logs‘ directory, applies search filters, and paginates the results based on the specified offset and limit. The returned data includes metadata such as file size and type.

- @search => Search term or query to filter files.
- @offset => Offset for pagination.
- @limit => Maximum number of files to retrieve.

Returns a list of files with metadata including ID, name, size, and type.

2.5.2 Get the full path of a file

```
function __files_getfile($file)
```

This function searches for a file by its name in the ‘data/logs‘ directory and returns its full path if found. Otherwise, it returns an empty string.

- @file => Name of the file to search for.

Returns the full path of the file or an empty string if not found.

2.5.3 Check if a file exists

```
function __files_check($file)
```

This function checks whether a file exists in the ‘data/logs‘ directory by searching for its name.

- @file => Name of the file to check.

Returns ‘true‘ if the file exists, otherwise ‘false‘.

2.5.4 View the contents of a file

```
function __files_view($file)
```

This function retrieves the contents of a file from the ‘data/logs‘ directory. It supports both regular files and gzip-compressed files. The returned data includes the file name, size, type, and content.

- @file => Name of the file to view.

Returns metadata and content of the file. If the file is not found, it returns an error response.

2.6 Help feature

```
apps/common/php/help.php
```

This file contains the help functions used by SaltOS

2.6.1 Detect Help File

```
function detect_help_file($app, $lang)
```

This function is intended to return the name of the pdf file used as help for the app and lang, they use 4 checks to search the correct file that must to returns, the first in to search in the app folder for the specified lang, otherwise search for some othe lang, and if no app file is found, then the same process is used for the notfound.pdf file.

- @app => the application to search
- @lang => the prefered lang to search

2.7 Matrix functions

```
apps/common/php/matrix.php
```

This file contains all the functions required by the Excel widget for handling version and log data in a matrix format.

2.7.1 Create matrix for version data

```
function make_matrix_version($app, $id)
```

This function generates a matrix structure for displaying version-related data in the Excel widget, including headers, data formatting, and cell attributes.

2.7.2 Create matrix for log data

```
function make_matrix_log($app, $id)
```

This function processes log data and generates headers and formatting information for integration into the Excel widget.

2.8 Helper file for the order feature

```
apps/common/php/order.php
```

2.8.1 Get Order Fields

```
function get_order_fields($app)
```

Get Orderable Fields from the XML Definition, to do it, parses a given XML file and extracts the list of field names that are marked as orderable.

- @app => app to search the xml file that contains the layout definition.

Returns an array of field names that have the 'order' attribute enabled.

2.9 PDF helper file

```
apps/common/php/pdf.php
```

2.9.1 Build the expected PDF template file path for the given app.

```
function detect_pdf_file($app)
```

Given an application ID, this function resolves the folder name using 'detect_app_folder($app)' and returns the full relative path to its associated PDF XML definition file, expected at:

apps/<folder>/xml/<app>_pdf.xml

This path is used to define the PDF layout when exporting records from the app.

- @app => The application code (e.g., 'invoices', 'quotes')

Return the relative path to the PDF XML file

2.9.2 Check if the PDF layout file exists for the given app.

```
function exists_pdf_file($app)
```

This function uses 'detect_pdf_file()' to resolve the expected path of the XML file used to generate PDF output for a given app, and returns whether the file actually exists.

- @app => The application code (e.g., 'invoices', 'quotes')

Return true if the PDF file exists, false otherwise

3 Crm

3.1 Quotes application

```
apps/crm/js/quotes.js
```

This application provides core functionalities for managing quotes, including creating, editing, viewing, and exporting quotes in PDF format.

3.1.1 Main object

```
saltos.quotes = {};
```

This object contains all SaltOS code related to quote operations.

3.1.2 Fix tax dropdown visibility.

```
saltos.quotes.onload = instance
```

Reset the parent's overflow property to prevent the tax dropdown from being clipped or hidden inside the container.

3.1.3 Lines columns

```
saltos.quotes.cols = {
```

This object defines the position of each column in the lines matrix, the tax and the withholding columns are always present in the matrix but are hidden when the mode defined by the company is not by line.

3.1.4 Types

```
saltos.quotes.types = {
```

This object defines the value used by each type in the taxes master, the modes defined by the company for each type can be 0 (not used), 1 (one by document) or 2 (by line).

3.1.5 Handle and recalculate quote line totals and tax breakdowns after user edits.

```
saltos.quotes.onchange = (instance, cell, x, y, value)
```

This function is triggered after the user modifies any cell in the "lines" matrix (quote lines). It performs the following operations:

- Validates and normalizes quantity, price, and discount values:
  - Ensures quantity, price and discount are numeric.
  - Defaults discount to 0 if empty but qty and price are valid.
- Sets the tax and the withholding of the new lines:
  - Uses the value of the document select when the mode is one by document.
  - Uses the value of the default register when the mode is by line.
  - The lines that already have a total are never modified.
- Calculates the line total as: ‘qty × price × (1 - discount%)‘, rounded to 2 decimals.
- Updates the lines grid to reflect computed values.
- Recomputes the taxes matrix:
  - Aggregates taxable bases per type (tax or withholding) and value.
  - Computes the amount for each group: ‘base × (value% / 100)‘.
- Rounds all results to 2 decimals and updates the taxes grid.
- Computes the totals matrix:
  - ‘subtotal‘ = sum of all line totals
  - ‘tax‘ = sum of all tax amounts
  - ‘withholding‘ = sum of all withholding amounts
  - ‘total‘ = subtotal + tax - withholding
  - Results are rounded and shown in the totals grid.
- @instance => The spreadsheet instance that triggers the change
- @cell => The cell that has changed
- @x => The column of the cell
- @y => The row of the cell
- @value => The new value of the cell

3.1.6 Apply the tax or the withholding of the document to all lines

```
saltos.quotes.onselect = type
```

This function is triggered when the user changes the select used by the one by document mode, it copies the selected value to all computed lines and recalculates the taxes and the totals, this allow to use the same structure of data for all modes.

- @type => The type of the select that has changed, can be tax or withholding

3.2 Reverses the process of ‘make_matrix_data()‘ for the ‘quotes‘ app.

```
apps/crm/php/quotes.php
```

This function takes a previously generated matrix-style array of quote data (grouped as lines, taxes, and totals), compares it with the current values in the database, and builds a minimal diff-like array that includes only the fields that have changed, been removed, or added.

3.2.1 unmake_matrix_data

```
function unmake_matrix_data($json, $quote_id)
```

Reverse the matrix-encoded quote data to a diff-style data array.

- @json => The matrix-encoded quote data (from make_matrix_data)
- @quote_id => The quote ID used to retrieve the original data

Return a diff array representing only the differences between the provided data and the current state of the database.

3.2.2 Get next code

```
function get_next_code($table, $field, $prefix)
```

This function returns the next code of a serie, the codes are composed by a prefix that contains the year and a correlative number, the number is computed using the numeric value of the codes that begin by the prefix, by this reason the serie is restarted when the year changes.

- @table => the table that contains the codes
- @field => the field that contains the codes
- @prefix => the prefix of the serie, for example PF2026-

3.2.3 Generates or completes quote and final quote metadata.

```
function set_quote($json, $quote_id)
```

This function ensures that the given quote JSON structure has appropriate values for:

- ‘code‘, ‘date‘ and ‘valid_until‘ (if not provided or if it’s a new quote)

Rules:

- If no ‘quote_id‘ is given or ‘code‘ is not defined, a new code number is generated.
- @json => Quote data to process and complete.
- @quote_id => ID of the quote, used to retrieve current status from DB.

Return the updated JSON with appropriate codes and dates filled in.

4 Dashboard

4.1 Dashboard application

```
apps/dashboard/js/dashboard.js
```

This module implements the typical features associated with a dashboard, allowing dynamic interaction with various elements and functionalities.

4.1.1 Initialization of dashboard

```
saltos.form.dashboard = data
```

This method sets up listeners, configures the catalog layout, and initializes the dashboard widgets based on user-defined or default configurations.

4.2 Dashboard and navbar generation logic.

```
apps/dashboard/php/dashboard.php
```

This file contains helper functions used to build the SaltOS dashboard and navbar dynamically based on the applications configured in the system and the permissions of the current user.

Functions included:

- __dashboard_helper(): Generates the full dashboard widget layout (alerts, buttons, separators)
- __dashboard_config(): Applies user-specific configuration to customize the dashboard layout
- __navbar_helper(): Builds the application menu structure for the top navigation bar

These functions are used internally by the SaltOS UI rendering engine to generate the initial layout and navigation menus seen by each user on login.

4.2.1 Build the default dashboard layout with groups and application buttons.

```
function __dashboard_helper()
```

This function retrieves all active applications from ‘tbl_apps‘, groups them by ‘group‘, filters them by user permissions (menu access), and then formats them into a widget-based structure suitable for rendering on the dashboard, including alerts, buttons, and separators.

Each group of apps is preceded by an alert (group title and description), followed by buttons for each app, and a horizontal rule (‘<hr>‘) at the end.

- @return array An array of widgets (alerts, buttons, hr) to render the dashboard

4.2.2 Build the application menu for the navbar.

```
function __navbar_helper()
```

This function generates the main application menu for the navbar by:

- Fetching all active apps grouped by ‘group‘
- Filtering them by user permissions
- Mapping the group metadata from ‘tbl_apps_groups‘
- Creating a linear structure of ‘<item>‘ elements including:
  - Group labels (disabled items)
  - App entries with onclick handlers
  - A divider after each group

The result is an array of menu items that represents the full application navigation tree grouped visually in the UI.

- @return array An array of navbar ‘<item>‘ definitions

5 Emails

5.1 Email application

```
apps/emails/js/driver.js
```

This application implements the typical features associated with email functionality, including templates and initialization processes for handling email-related tasks.

5.1.1 Driver type2emails object

```
saltos.driver.__types.type2emails = {};
```

This object stores the functions and properties used by the email driver, providing the necessary methods for managing email operations.

5.1.2 Create type2emails template

```
saltos.driver.__types.type2emails.template = arg
```

This function generates a template for the email driver, configuring specific attributes and layout modifications. It sets the 'type' attribute to 'type2emails' and adjusts the layout of the corresponding element for proper display.

5.1.3 Initialization, open, and close handlers for type2emails

```
saltos.driver.__types.type2emails.init = saltos.driver.__types.type2modal.init; //
    Inherits initialization
```

These methods inherit their implementations from the 'type2modal' driver, allowing reuse of core functionality for initializing, opening, and closing email resources.

5.2 Email application

```
apps/emails/js/emails.js
```

This application implements typical features associated with email functionality, such as initialization, server actions, email deletion, sending, setting states, handling signatures, generating and downloading PDFs, viewing email sources, replying and forwarding.

5.2.1 Main object

```
saltos.emails = {};
```

This object contains all SaltOS code related to email operations.

5.2.2 Initialize the email application

```
saltos.emails.init = arg
```

This method handles initialization tasks for different views (‘create‘, ‘view‘, ‘close‘, ‘list‘) in the email application. It manages UI updates, visibility behaviors, and email state changes.

- @param {string} arg Specifies the type of view or action to initialize.

5.2.3 Perform server-related actions for emails

```
saltos.emails.server = ()
```

This method triggers server-side actions for emails and displays the responses in toasts.

5.2.4 Delete selected emails

```
saltos.emails.delete1 = ()
```

This method deletes multiple emails selected by the user, after showing a confirmation modal.

5.2.5 Delete a single email

```
saltos.emails.delete2 = ()
```

This method deletes the currently viewed email after showing a confirmation modal.

5.2.6 Send an email

```
saltos.emails.send = ()
```

This method validates the required fields and sends the email data to the server. It handles responses and clears unsaved data on success.

5.2.7 Set email state

```
saltos.emails.setter = what
```

This function updates the state of a specific email identified by its ID. The updated state is sent to the server, and the UI is refreshed afterward.

- @param {string} what Specifies the state to set for the email.

5.2.8 Add signature to email

```
saltos.emails.signature = ()
```

This function applies the email signature based on the selected account and its related data. The signature is retrieved from the server and updated in the email form.

5.2.9 View email source

```
saltos.emails.source = ()
```

This function opens the raw source of the selected email for viewing in the driver.

5.2.10 Reply to an email

```
saltos.emails.reply = ()
```

This function opens the reply form for the currently selected email.

5.2.11 Reply to all recipients of an email

```
saltos.emails.replyall = ()
```

This function opens the reply-all form for the currently selected email, including all recipients.

5.2.12 Forward an email

```
saltos.emails.forward = ()
```

This function opens the forward form for the currently selected email.

5.2.13 Driver search

```
saltos.driver.search = arg
```

This function implement the search feature associated to tables and lists using the filters fields

Additionally, translates the ‘body_text‘ field of each result before rendering the list.

- @arg => unused at this scope

5.3 Email Query Builder

```
apps/emails/php/filter.php
```

This function builds an SQL WHERE query for filtering emails based on various criteria such as account, fields, search terms, dates, and specific email states.

5.3.1 Generate WHERE query for emails

```
function make_where_query_emails($json)
```

This function takes a JSON input and generates a dynamic SQL WHERE clause to filter emails. The query is constructed based on several optional parameters such as account ID, search fields, dates, and flags indicating specific states (e.g., new emails, waiting emails, spam).

- @json => Input parameters for generating the query.

Returns the dynamically constructed SQL WHERE clause.

5.4 Get email library

```
apps/emails/php/getmail.php
```

This library provides the necesary functions to download, parse and manage emails.

5.4.1 Requires section

```
require_once 'apps/emails/php/mailparse.php';
```

This requires loads the external libraries needed to run this library.

5.4.2 Defines section

```
// phpcs:disable Generic.Files.LineLength
```

This defines allow to define some useful standards to do html pages and more.

5.4.3 Remove all body

```
function __getmail_removebody($array)
```

This function removes the body entry in the array, it is only for debug purposes

- @aarray => The array that you want to process

5.4.4 Process message

```
function __getmail_processmessage($disp, $type)
```

This function returns a boolean that identify if the disposition and the type allow the node to be processed.

- @disp => disposition, can be inline or attachment
- @type => type, can be message, html or plain

5.4.5 Process plain html

```
function __getmail_processplainhtml($disp, $type)
```

This function returns a boolean that identify if the disposition and the type allow the node to be processed.

- @disp => disposition, can be inline or attachment
- @type => type, can be message, html or plain

5.4.6 Process file

```
function __getmail_processfile($disp, $type)
```

This function returns a boolean that identify if the disposition and the type allow the node to be processed.

- @disp => disposition, can be inline or attachment
- @type => type, can be message, html or plain

5.4.7 Check permissions

```
function __getmail_checkperm($id)
```

This function allow to check if the current user has permissions to view the message identified by the id argument

- @id => id of the email

5.4.8 Get GZ File

```
function __getmail_gzfile($id)
```

This function returns the file that contains the email

- @id => id of the email

5.4.9 Get source

```
function __getmail_getsource($id)
```

This function returns the original RFC822 message as string

- @id => id of the email

Notes:

This function returns the email source without the base64 attachments

5.4.10 Mime decode protected

```
function __getmail_mime_decode_protected($input)
```

This function decodes the input string that contains the RFC822 message using the mime_parser_class to do it, and returns the decoded array.

- @input => the RFC822 string that contains the message

5.4.11 Get mime

```
function __getmail_getmime($id)
```

This function returns the decoded array of the email identified by the id argument, to do this more optimal, this function uses an internal cache file to improve the performance for repeated executions.

- @id => id of the email

5.4.12 Get Node

```
function __getmail_getnode($path, $array)
```

This function returns the node using a xpath notation

- @path => xpath that identify the desired path that must to be returned
- @array => the decoded message in an array format

5.4.13 Get type

```
function __getmail_gettype($array)
```

This function tries to unify the differents content-type to standarize it into the follow formats: html, plain, messsage, alternative, multipart or other.

- @array => the decoded message in an array format

5.4.14 Get disposition

```
function __getmail_getdisposition($array)
```

This function tries to unify the differents content-dispoaition to standarize it into the follow formats: attachment, inline or other.

- @array => the decoded message in an array format

5.4.15 Get files

```
function __getmail_getfiles($array, $level = 0)
```

This function returns an array with the attachment files of the message

- @array => the decoded message in an array format
- @level => this parameter is internally used to detect recursion

5.4.16 Get info

```
function __getmail_getinfo($array)
```

Returns all information of the decoded message in a structured format

- @array => the decoded message in an array format

5.4.17 Fix string

```
function __getmail_fixstring($arg)
```

This function is a helper used by all functions that pcoesses the headers of the decoded message.

- @arg => the string that must to be checked and fixed if needed

5.4.18 Get text body

```
function __getmail_gettextbody($array, $level = 0)
```

This function returns all text body concatenated as an unique string

- @array => the decoded message in an array format
- @level => this parameter is internally used to detect recursion

5.4.19 Get full body

```
function __getmail_getfullbody($array)
```

This function returns all body and attachments information as an array

- @array => the decoded message in an array format

5.4.20 Get cid

```
function __getmail_getcid($array, $hash)
```

This function returns the requested attachment indentified by the hash argument

- @array => the decoded message in an array format
- @hash => the hash of the content requested

5.4.21 Insert

```
function __getmail_insert(
```

This function do the insert in the app_emails table, and

- @file => the gzfile that contains the message in RFC822 format
- @messageid => the id of the message (account_id/uidl)
- @state_new => the 0/1 that sets the state new flag
- @state_reply => the 0/1 that sets the state reply flag
- @state_forward => the 0/1 that sets the state forward flag
- @state_wait => the 0/1 that sets the state wait flag
- @id_correo => the id of the related email (used to create relations between emails)
- @is_outbox => the 0/1 that sets the is outbox flag
- @state_sent => the 0/1 that sets the state sent flag
- @state_error => the string that contains the error (if exists an error)

5.4.22 Update

```
function __getmail_update($field, $value, $id)
```

This function updates the field with the value of the app_emails for the register identified by the id argument.

- @field => field that you want to update
- @value => value that you want to set
- @id => id of the register to do the update

5.4.23 Add bcc

```
function __getmail_add_bcc($id, $bcc)
```

This function adds the bbc to the database, this is because the messages does not contains the bcc field (is hidden in theory), and only is available if the current execution is the sender of the message.

- @id => id of the email
- @bcc => an array with the addresses of the emails

5.4.24 Gzfile size

```
function gzfilesize($filename)
```

This function is copied from http://php.net/manual/es/function.gzread.php#110078 and allow to know the file size of the file after a gzip descompression.

- @filename => the gzip filename that you want to know the size

5.4.25 Get email body

```
function getmail_body($id, $images = false)
```

This function returns the string that contains the body of the email intended to be rendered in an iframe, for example

- @id => id of the email

5.4.26 Generate formatted email header information

```
function __getmail_head_helper($decoded, $email_id)
```

This function extracts and formats header details from the decoded email structure, including sender, recipients, date, subject, and attachments. It handles edge cases such as missing data, and ensures that all information is presented in a structured and readable format. HTML encoding is applied to ensure safe display within a web interface.

- @decoded => Decoded structure of the email, containing metadata and header information.
- @email_id => ID of the email to retrieve additional account-specific details.

Returns a formatted HTML string with the email header information.

5.4.27 Extract and format the body of an email

```
function __getmail_body_helper($decoded, $images = false)
```

This function processes the decoded content of an email, extracting the plain text and HTML bodies while applying necessary transformations for safe and structured display. It handles inline images, styles, and embedded attachments, ensuring compatibility and security.

- @decoded => Decoded structure of the email, typically containing nodes with metadata and body content.
- @images => Specifies whether inline images should be embedded directly into the content.

Returns the formatted email body, including plain text, HTML, and inline attachments.

5.4.28 Get email source

```
function getmail_source($id)
```

This function returns the string that contains the source of the email intended to be rendered in an iframe, for example

- @id => id of the email

5.4.29 Get email files

```
function getmail_files($id)
```

This function returns an arryy that contains the files of the email intended to be rendered in an table, for example

- @id => id of the email

5.4.30 Get cid

```
function getmail_cid($id, $cid)
```

This function returns the requested attachment indentified by the cid argument

- @id => id of the email
- @cid => the cid of the content requested

5.4.31 Is outbox

```
function getmail_field($field, $id)
```

Returns the field of the email identified by the id argument

- @field => field requested
- @id => id of the email

5.4.32 Server

```
function getmail_server()
```

This function implements the old getmail action of the old saltos.

5.4.33 Delete

```
function getmail_delete($ids)
```

This function implements the old delete action of the old saltos.

- @ids => array with the emails id

5.4.34 Get viewpdf

```
function getmail_viewpdf($id, $cid)
```

This function returns the requested attachment indentified by the cid argument in a pdf format for the viewpdf widget

- @id => id of the email
- @cid => the cid of the content requested

5.4.35 Get download

```
function getmail_download($id, $cid)
```

This function returns the requested attachment indentified by the cid argument in an array format for the download feature

- @id => id of the email
- @cid => the cid of the content requested

5.4.36 Update email states

```
function getmail_setter($ids, $what)
```

This function modifies the state of emails (e.g., ‘new‘, ‘wait‘, or ‘spam‘) based on the provided IDs and action. It validates user permissions for the selected emails before updating their states. The function calculates the number of records to be modified and updates the database accordingly.

- @ids => Comma-separated list of email IDs to be updated.
- @what => Specifies the state change in the format ‘key=value‘ (e.g., ‘new=0‘).

Returns a response message indicating the number of emails modified.

5.4.37 Generate PDF from emails

```
function getmail_pdf($ids)
```

This function generates PDF files for the provided email IDs using either the ‘chromium‘ command or a fallback library. If multiple emails are selected, their PDFs are merged into a single file. It validates user permissions for accessing the emails and manages caching for performance optimization.

- @ids => List or comma-separated string of email IDs to generate PDFs for.

Returns metadata of the generated PDF file, including its name, type, and base64-encoded data.

5.4.38 Generate iframe content with secure policies

```
function __iframe_srcdoc_helper($html)
```

This function generates the ‘srcdoc‘ content for an iframe element, embedding the provided HTML string. It ensures the content is styled using predefined fonts and enforces a strict Content Security Policy (CSP) to limit the sources for styles, fonts, and images. This helps maintain security and compatibility within the iframe.

- @html => The HTML content to be embedded in the iframe.

Return a complete HTML document string suitable for ‘srcdoc‘ use in an iframe.

5.5 Html helper module

```
apps/emails/php/html.php
```

This file contain useful html helper functions

5.5.1 Defines section

```
define('__GIF_IMAGE__', '
   ywAAAAACgABAAACA4SPBQA7');
```

This defines allow to define some useful needed resources by this file.

5.5.2 Remove Script Tag

```
function remove_script_tag($html)
```

This function tries to remove all <script> tags of the string

- @temp => the string that you want to process

5.5.3 Remove Style Tag

```
function remove_style_tag($html)
```

This function tries to remove all <style> tags of the string

- @temp => the string that you want to process

5.5.4 Remove Comment Tag

```
function remove_comment_tag($html)
```

This function tries to remove all <!- -> tags of the string

- @temp => the string that you want to process

5.5.5 Remove Meta Tag

```
function remove_meta_tag($html)
```

This function tries to remove all <meta > tags of the string

- @temp => the string that you want to process

5.5.6 Remove Link Tag

```
function remove_link_tag($html)
```

This function tries to remove all <link > tags of the string

- @temp => the string that you want to process

5.5.7 Inline Img Tag

```
function inline_img_tag($html)
```

This function tries to convert all imgs to an inline imgs

- @temp => the string that you want to process

5.5.8 Inline Img Style

```
function inline_img_style($html)
```

This function tries to convert all imgs to an inline imgs

- @temp => the string that you want to process

5.5.9 Inline Img Background

```
function inline_img_background($html)
```

This function tries to convert all imgs to an inline imgs

- @temp => the string that you want to process

5.5.10 Inline Image Helper

```
function __inline_img_helper($src)
```

This function fetches and processes image URLs, ensuring compatibility and resizing where necessary. It also manages caching and handles errors gracefully.

- @src => Image source URL or path.

Returns the inline image data or a placeholder GIF in case of errors.

5.5.11 Extract Inline Images from Tags

```
function extract_img_tag($html)
```

This function extracts inline images from HTML ‘<img>‘ tags and replaces their ‘src‘ attributes with ‘cid‘ references for email compatibility.

- @html => HTML content to parse.

Returns the modified HTML content and an array of image files.

5.5.12 Extract Inline Images from Styles

```
function extract_img_style($html)
```

This function extracts images embedded in CSS ‘style‘ attributes and replaces them with ‘cid‘ references for email compatibility.

- @html => HTML content to parse.

Returns the modified HTML content and an array of image files.

5.5.13 Extract Inline Images from Backgrounds

```
function extract_img_background($html)
```

This function extracts images embedded in HTML ‘background‘ attributes and replaces them with ‘cid‘ references for email compatibility.

- @html => HTML content to parse.

Returns the modified HTML content and an array of image files.

5.5.14 Fix Image Tags

```
function fix_img_tag($html)
```

This function processes ‘<img>‘ tags in the given HTML to replace non-inline image ‘src‘ attributes with a placeholder image (‘GIF_IMAGE‘), ensuring compatibility and avoiding external dependencies.

- @html => The HTML content to process.

Returns the updated HTML content with fixed image tags.

5.5.15 Fix Image Styles

```
function fix_img_style($html)
```

This function processes CSS ‘style‘ attributes in HTML elements to replace non-inline image URLs within ‘url()‘ properties with a placeholder image (‘GIF_IMAGE‘).

- @html => The HTML content to process.

Returns the updated HTML content with fixed image styles.

5.5.16 Fix Image Backgrounds

```
function fix_img_background($html)
```

This function processes ‘background‘ attributes in HTML elements to replace non-inline image URLs with a placeholder image (‘GIF_IMAGE‘).

- @html => The HTML content to process.

Returns the updated HTML content with fixed image backgrounds.

5.5.17 Fix Base64-Encoded Files

```
function fix_file_b64($html)
```

This function replaces inline base64-encoded file data in HTML with the corresponding cached image content, ensuring that embedded images are properly handled.

- @html => The HTML content to process.

Return the updated HTML content with fixed base64-encoded files.

Notes:

We are using '{48}' to match base64 data of 48 bytes, which decodes into a 36-byte string that matches the MD5 hash style used in cached files (32 bytes plus 4-byte '.b64' extension).

5.6 Make indexing action

```
apps/emails/php/indexing.php
```

This fie contains useful functions related to the indexing action that internally uses the mroonga engine to search in the fulltext string generated by this action

5.7 Mailparse helper module

```
apps/emails/php/mailparse.php
```

This file provides the subset of functions of the 'mailparse' PECL extension that mime_parser_class.php actually uses, intended to be used by setups that can not install this extension. Backed by the pure PHP composer package zbateson/mail-mime-parser.

Scope: only mailparse_msg_create, mailparse_msg_parse, mailparse_msg_get_structure, mailparse_msg_get_part, mailparse_msg_g mailparse_msg_extract_part, mailparse_msg_free and mailparse_rfc822_parse_addresses are provided, because those are the only ones consumed by mime_parser_class.php. The 'headers'/'content-type'/'disposition-filename'/'content-name' keys of mailparse_msg_get_part_data() are the only ones populated (the real extension returns a richer array).

Note: unlike the real extension, the Body extracted here is charset-normalized to UTF-8 by the underlying library (it only reverses the Content-Transfer-Encoding, mailparse leaves the original charset bytes as-is). Review any code that re-converts Body's charset by hand before relying on this polyfill in production.

Three known, deliberately-not-chased differences, verified harmless against a large real-inbox sample compared via utest/test_mailp

- mailparse_rfc822_parse_addresses(): for an address with no display name (e.g. a bare "user@host", no "Name <user@host>"), the real extension echoes the address itself back as 'display' and adds an 'is_group' key (always false in every sample seen). The library leaves 'display' as '' and this polyfill never sets 'is_group' at all. Neither is chased because mime_parser_class.php already collapses both shapes to the same result: it drops 'display' via 'strcasecmp($disp, $email) !== 0' (true only when there's a real name) and never reads 'is_group' at all.
- Trailing whitespace on a header's value: the library trims it during its own header parsing (Parser/HeaderParserService.php), before it is exposed through any public method, so it can not be recovered here. The real extension keeps it. Purely cosmetic (e.g. a Subject ending in one extra space), never observed to affect parsing logic.
- Two adjacent RFC 2047 encoded-words in a display name (a name split across encoded-words for line-length reasons): the real extension's mb_decode_mimeheader() collapses the whitespace between them, which can glue two words together (eg. "SerniRibó"). The library decodes each encoded-word independently and keeps the whitespace between them (eg. "Serni Ribó"), which reads as the more correct behaviour and is what this polyfill produces.

Layout: the actual polyfill logic lives in the __mailparse_*_helper() functions/class below (never guarded: they can't collide with anything the extension provides). Every mailparse_* function is just a thin delegate to its same-named helper, grouped at the end of this file and guarded with function_exists() so it's only defined when the real extension is missing. That split exists so utest/test_mailparse.php can call the helper directly and exercise this polyfill's actual code even on machines that do have the native extension installed, where the plain mailparse_* names are already taken by the extension and can't be used to reach this file's code.

5.7.1 Internal handle used by the functions below to keep the parsed

```
final class __mailparse_handle_helper
```

together, emulating the resource returned by mailparse_msg_create().

Not part of the mailparse extension's API: naming follows the project's convention for internal/helper symbols (leading "__", trailing "_helper"), same as __getmail_head_helper() elsewhere in this app, so it reads at a glance as "not a drop-in replacement".

5.7.2 Mailparse Index Parts Helper

```
function __mailparse_index_parts_helper($part, string $id, array &$out): void
```

Walks the part tree returned by zbateson/mail-mime-parser and assigns it mailparse-style dot-notation ids ("1", "1.1", "1.2", "1.1.1"...), matching what mime_parser_class.php expects to find in the array returned by mailparse_msg_get_structure().

A message/rfc822 part (a forwarded email carried as an attachment) does not expose its inner structure through getChildParts(): the library treats it as an opaque leaf whose content happens to be a full embedded message. mailparse_msg_get_structure() from the real extension does recurse into it, so we parse that embedded content separately here and splice the result in as this part's single child, matching the native behaviour.

- @part => current part being indexed
- @id => dot-notation id to assign to this part
- @out => output map, id => part, built by reference

5.7.3 Mailparse Unfold Helper

```
function __mailparse_unfold_helper(string $value): string
```

getRawValue() de la libreria conserva el folding (RFC 5322) de la cabecera tal cual aparece en el mensaje original (con el CRLF/LF antes del espacio de continuacion). La extension mailparse real desdobla la cabecera (unfolding), quitando esos saltos de linea y dejandola en una sola linea. Replicamos ese mismo comportamiento aqui para que el resultado sea comparable.

- @value => valor crudo de la cabecera, potencialmente con folding

5.7.4 Mailparse Encode Display Helper

```
function __mailparse_encode_display_helper(string $name): string
```

getName() de la libreria ya devuelve el nombre mostrado con el RFC 2047 (encoded-word) decodificado a UTF-8. La extension mailparse real, en cambio, devuelve ese display name tal cual aparece en el mensaje (todavia codificado si el remitente uso encoded-words), ya que quien llama a mailparse_rfc822_parse_addresses() en SaltOS (mime_parser_class) pasa siempre ese valor por mb_decode_mimeheader() despues. Para mantener el mismo contrato aqui, si el nombre ya decodificado contiene bytes no-ASCII lo volvemos a codificar como encoded-word: mb_decode_mimeheader() no toca el texto ASCII plano pero corrompe UTF-8 crudo que no venga envuelto en "=?...?=".

Se reutiliza igual para disposition-filename/content-name, que getHeaderParameter() tambien devuelve ya decodificados y que mime_parser_class.php tambien pasa por mb_decode_mimeheader().

- @name => nombre ya decodificado devuelto por AddressPart::getName() o por getHeaderParameter()

5.7.5 Mailparse Msg Create Helper

```
function __mailparse_msg_create_helper()
```

- @path => adjust to wherever your lib/ layout puts this package, following the same convention as lib/yaml/vendor/autoload.php

5.7.6 Mailparse Msg Parse Helper

```
function __mailparse_msg_parse_helper($msg, string $data): bool
```

- @msg => handle returned by mailparse_msg_create()
- @data => the full raw message

5.7.7 Mailparse Msg Get Structure Helper

```
function __mailparse_msg_get_structure_helper($msg): array
```

- @msg => handle previously passed to mailparse_msg_parse()

5.7.8 Mailparse Msg Get Part Helper

```
function __mailparse_msg_get_part_helper($msg, string $partId)
```

- @msg => handle previously passed to mailparse_msg_parse()
- @partId => one of the ids returned by mailparse_msg_get_structure()

5.7.9 Mailparse Msg Get Part Data Helper

```
function __mailparse_msg_get_part_data_helper($part): array
```

Only fills the keys that mime_parser_class.php reads: headers, content-type, disposition-filename and content-name.

- @part => part returned by mailparse_msg_get_part()

5.7.10 Mailparse Msg Extract Part Helper

```
function __mailparse_msg_extract_part_helper($part, string $data, $callback = null):
    string
```

- @part => part returned by mailparse_msg_get_part()
- @data => full raw message (unused here, the library already keeps its own reference to the content while parsing)
- @callback => decoding callback (unused, always fully decoded)

5.7.11 Mailparse Msg Free Helper

```
function __mailparse_msg_free_helper($msg): bool
```

- @msg => handle previously returned by mailparse_msg_create()

5.7.12 Mailparse Rfc822 Parse Addresses Helper

```
function __mailparse_rfc822_parse_addresses_helper(string $addresses): array
```

Parses a raw address header value (e.g. the content of a "To:" header) into a list of 'display' => ..., 'address' =>.

Builds a throwaway one-header message under a made-up header name so the value is run through the library's own RFC-822 address consumer, then forces it to be read as an AddressHeader via getHeaderAs() regardless of whether that header name is one of the library's pre-registered address headers.

- @addresses => raw header value, e.g. '"Doe, John" <john@example.com>'

5.7.13 Thin mailparse_* delegates: only defined when the real extension is

__mailparse_*_helper() above.

5.7.14 Mailparse Msg Create

```
function mailparse_msg_create()
```

5.7.15 Mailparse Msg Parse

```
function mailparse_msg_parse($msg, string $data): bool
```

5.7.16 Mailparse Msg Get Structure

```
function mailparse_msg_get_structure($msg): array
```

5.7.17 Mailparse Msg Get Part

```
function mailparse_msg_get_part($msg, string $partId)
```

5.7.18 Mailparse Msg Get Part Data

```
function mailparse_msg_get_part_data($part): array
```

5.7.19 Mailparse Msg Extract Part

```
function mailparse_msg_extract_part($part, string $data, $callback = null): string
```

5.7.20 Mailparse Msg Free

```
function mailparse_msg_free($msg): bool
```

5.7.21 Mailparse Rfc822 Parse Addresses

```
function mailparse_rfc822_parse_addresses(string $addresses): array
```

5.8 SaltOS MIME Parser Component

```
apps/emails/php/mime_parser.php
```

PURPOSE: Replaces legacy phpclasses.org implementations (‘mime_parser‘ and ‘rfc822_addresses‘) with native PHP ‘mailparse‘ functionality for improved performance and maintenance.

COMPATIBILITY: Maintains output structure (‘Headers‘, ‘DecodedHeaders‘, ‘ExtractedAddresses‘, ‘Body‘, ‘BodyLength‘, ‘Parts‘) to ensure seamless integration with existing SaltOS codebase.

ARCHITECTURE:

- Recursive tree building via ‘mailparse_msg_get_structure()‘.
- Supports memory (‘Data‘) or stream-based (‘File‘) input.
- Optional body extraction for non-multipart nodes.

5.8.1 - @param array $input Must be an array with 'File' or 'Data'

```
public function Decode($input, array &$decoded): bool
```

- @return bool

5.8.2 - @param resource $msg mailparse message handle

```
private function buildNode($msg, string $partId, array $struct, string $raw, bool
    $decodeBodies): array
```

- @param array $struct cached full structure
- @param string $raw full message in memory
- @param bool $decodeBodies

5.9 SaltOS POP3 Client Component

```
apps/emails/php/pop3.php
```

PURPOSE: Replaces legacy phpclasses.org POP3 implementations with a modern, cURL-based architecture to eliminate dependency on obsolete external code.

ARCHITECTURE & DESIGN:

- Stateless per-command operation: Executes verbs via fresh cURL transfers instead of maintaining persistent app-level TCP sessions.
- Transport: Supports implicit TLS (POP3S) or opportunistic STARTTLS.
- Centralized execution: 'exec()' handles protocol multi-line/single-line responses and error normalization.

SECURITY & CONFIGURATION:

- Warning: Certificate verification is currently disabled for compatibility. Production environments should enable peer/host verification.
- Requirement: PHP cURL extension (ext-curl).

5.9.1 Prepare the cURL handle and compute the base URL based on configuration.

```
public function Open()
```

This does not establish a persistent POP3 session; it only sets up the common cURL options used by exec().

- @return string Empty string on success; non-empty error message on failure.

5.9.2 Retrieve the server's UIDL listing and return a map of index => uidl.

```
public function ListMessages()
```

Behavior:

- Issues "UIDL" and parses the multi-line response.
- Skips status lines (+OK / -ERR) and returns only numbered entries.

Caveat:

- Indices are assigned by the server and can change between calls if messages are deleted. Consumers should prefer using UIDLs as stable ids.
- @return array<int,string>|string Array of msgIndex => on success; error string on failure.

5.9.3 Download a full message by server index using "RETR <index>".

```
public function GetMessage($index, &$out)
```

- @param int $index 1-based server message index.
- @param string|null $out Filled with the raw RFC 5322 message on success.
- @return string Empty string on success; non-empty error message on failure.

5.9.4 Issue "DELE <index>" to mark a message for deletion.

```
public function DeleteMessage($index)
```

Notes:

- Since this client operates per-command with independent connections, deletion is immediately applied server-side for that command.
- Be careful when chaining deletions by index; if indices shift between commands, you may delete unintended messages. Prefer resolving indices from UIDLs with ListMessages() right before deletion.
- @param int $index 1-based server message index.
- @return string Empty string on success; non-empty error message on failure.

5.9.5 Release the cURL handle. No explicit POP3 QUIT is required because each

```
public function Close()
```

- @return string Always returns empty string.

5.9.6 Execute a POP3 command using cURL.

```
// @phpstan-ignore parameterByRef.unusedType
```

Behavior:

- Sets CURLOPT_CUSTOMREQUEST to the given verb (e.g., "UIDL", "RETR 1", "DELE 2").
- When $withBody is true (multi-line commands like UIDL/LIST/RETR/TOP), fetches and returns the response body in $out.
- When $withBody is false (single-line commands like NOOP/DELE/CAPA), uses CURLOPT_NOBODY and forces a fresh connection to avoid hangs observed on some distributions (e.g., Debian 12).

Error handling:

- On transport failure (including a protocol-level "-ERR" reply, which libcurl's POP3 client rejects before returning it as body), returns a "cURL: ..." message.
- @param string $cmd POP3 command (verbatim), e.g., "UIDL", "RETR 1", "DELE 3".
- @param string|null $out Filled with response body when $withBody is true; otherwise ''.
- @param bool $withBody True for multi-line commands; false for single-line commands.
- @return string Empty string on success; non-empty error text on failure.

5.10 Send email library

```
apps/emails/php/sendmail.php
```

This library provides the necesary functions to send emails.

5.10.1 Used libraries

```
use PHPMailer\PHPMailer\PHPMailer;
```

This use loads the external libraries needed to run this library.

5.10.2 Sendmail

```
function sendmail($account_id, $to, $subject, $body, $files = '', $async = true)
```

This function send an email in synchronous and/or asynchronous mode

$account_id => the account id used to detect the source of the email $to => can be an string with the destination email or an array with the follow prefixes => to:, cc:, bcc:, crt:, priority:, sensitivity:, replyto $subject => the subject string $body => the body string $files => an array with files

Notes:

This function must return the last_id if success, and a string if error

5.10.3 Debugoutput helper

```
function __sendmail_debugoutput_helper($str, $level)
```

This function tries to echoed all debug information with the SMTP Error: prefix, with this feature we can capture smtp errors with more details like the error stored in $mail->ErrorInfo that always contains SMTP connect() failed, the signature of this function will accomplish the neested callback arguments

- @str => the debug trace string that can contain the errors
- @level => unused in this function

5.10.4 Parser

```
function __sendmail_parser($oldaddr)
```

This function gets an address and tries to detect the name part and the addr part of the argument. It's returns an array with two elements, the first is for the addr and the second is for the name.

- @oldaddr => the string that must to be processed

5.10.5 Message Id

```
function __sendmail_messageid($account_id, $from)
```

This function returns the message id for a new email, to do it, tries to detect the outbox directory, compute an aproximation to the newest value and checks that is unique in the system to prevent concurrence.

- @account_id => the account id used to send the new email
- @from => the from used to compute the crc32

5.10.6 Eml saver

```
function __sendmail_emlsaver($message, $messageid)
```

This function is intended to save the RFC822 message into the eml gzfile

- @message => the contents in RFC822 format of the message
- @messageid => the message id computed previously

5.10.7 Obj saver

```
function __sendmail_objsaver($mail, $messageid)
```

This function is intended to save the PHPMailer object into the obj file

- @mail => the PHPMailer object of the asynchronous transaction
- @messageid => the message id computed previously

5.10.8 Prepare email for sending

```
function sendmail_prepare($action, $email_id)
```

This function constructs the required parameters for sending an email, including sender details, recipients, subject, body, and attachments. The behavior varies based on the specified action (‘reply‘, ‘replyall‘, or ‘forward‘). It retrieves necessary data from the database and processes the email’s metadata to ensure compatibility with the given action.

- @action => Specifies the action to perform (‘reply‘, ‘replyall‘, or ‘forward‘).
- @email_id => ID of the email being replied to, forwarded, or used for metadata.

Return the prepared email parameters, including sender, recipients, subject, body, state, and attachments.

5.10.9 Perform email sending action

```
function sendmail_action($json, $action, $email_id)
```

This function handles the process of sending an email, including assembling email data, recipients, and attachments. It also manages inline images, updates email states, and cleans up temporary files after execution. The function supports multiple actions such as ‘reply‘, ‘replyall‘, and ‘forward‘.

- @json => JSON object containing email data (e.g., recipients, subject, body, attachments).
- @action => Specifies the action to perform (‘reply‘, ‘replyall‘, or ‘forward‘).
- @email_id => ID of the email being replied to, forwarded, or used for metadata.

Returns the status and message of the email action.

5.10.10 Send queued emails from the server

```
function sendmail_server()
```

This function processes and sends emails queued in the outbox. It uses a semaphore mechanism to prevent simultaneous execution, handles SMTP settings dynamically, updates email states, and manages errors during the sending process. The function is intended for use in cron jobs.

- @return array Returns an array of messages detailing the sending result, including errors and successes.

5.10.11 Retrieve email attachments

```
function sendmail_files($action, $email_id)
```

This function handles the retrieval of email attachments based on the specified action, such as ‘forward‘. It validates permissions, decodes the message, and stores attachments locally if not already uploaded. Finally, it retrieves a list of attachments from the upload table.

- @action => Specifies the email action (‘forward‘ in this case).
- @email_id => ID of the email to retrieve attachments from.

Returns a list of attachments with metadata including ID, app, name, size, type, and hash.

5.10.12 Update email signature, CC, and encryption state

```
function sendmail_signature($json)
```

This function updates the email body with a new signature, adjusts the CC list, and replaces the encryption state ('state_crt') based on the selected account.

- @json => JSON object containing email data and account information.

Return the updated email body, CC list, and encryption state.

6 Sales

6.1 Invoices application

```
apps/sales/js/invoices.js
```

This application provides core functionalities for managing invoices, including creating, editing, viewing, and exporting invoices in PDF format.

6.1.1 Main object

```
saltos.invoices = {};
```

This object contains all SaltOS code related to invoice operations.

6.1.2 Fix tax dropdown visibility.

```
saltos.invoices.onload = instance
```

Reset the parent's overflow property to prevent the tax dropdown from being clipped or hidden inside the container.

6.1.3 Lines columns

```
saltos.invoices.cols = {
```

This object defines the position of each column in the lines matrix, the tax and the withholding columns are always present in the matrix but are hidden when the mode defined by the company is not by line.

6.1.4 Types

```
saltos.invoices.types = {
```

This object defines the value used by each type in the taxes master, the modes defined by the company for each type can be 0 (not used), 1 (one by document) or 2 (by line).

6.1.5 Handle and recalculate invoice line totals and tax breakdowns after user edits.

```
saltos.invoices.onchange = (instance, cell, x, y, value)
```

This function is triggered after the user modifies any cell in the "lines" matrix (invoice lines). It performs the following operations:

- Validates and normalizes quantity, price, and discount values:

    - Ensures quantity, price and discount are numeric.
    - Defaults discount to 0 if empty but qty and price are valid.
- Sets the tax and the withholding of the new lines:
    - Uses the value of the document select when the mode is one by document.
    - Uses the value of the default register when the mode is by line.
    - The lines that already have a total are never modified.
- Calculates the line total as: 'qty × price × (1 - discount%)', rounded to 2 decimals.
- Updates the lines grid to reflect computed values.
- Recomputes the taxes matrix:
    - Aggregates taxable bases per type (tax or withholding) and value.
    - Computes the amount for each group: 'base × (value% / 100)'.
- Rounds all results to 2 decimals and updates the taxes grid.
- Computes the totals matrix:
    - 'subtotal' = sum of all line totals
    - 'tax' = sum of all tax amounts
    - 'withholding' = sum of all withholding amounts
    - 'total' = subtotal + tax - withholding
    - Results are rounded and shown in the totals grid.
- @instance => The spreadsheet instance that triggers the change
- @cell => The cell that has changed
- @x => The column of the cell
- @y => The row of the cell
- @value => The new value of the cell

6.1.6 Apply the tax or the withholding of the document to all lines

```
saltos.invoices.onselect = type
```

This function is triggered when the user changes the select used by the one by document mode, it copies the selected value to all computed lines and recalculates the taxes and the totals, this allow to use the same structure of data for all modes.

- @type => The type of the select that has changed, can be tax or withholding

6.2 ChartJS widgets for invoices

```
apps/sales/php/echarts.php
```

This file contains helper functions that generate Chart.js-compatible datasets for visualizing invoice-related data within SaltOS4.

Each function returns an associative array with 'labels' and 'datasets', directly usable by '<echarts>' widgets in the XML layout system.

The metrics provided include:

- Daily total invoicing
- Daily average invoice value

- Top 5 customers by total invoiced
- Count of paid vs unpaid invoices
- Average number of days to get paid

All functions rely on data from the ‘app_invoices‘ table and assume only closed invoices (‘is_closed = 1‘) are relevant.

6.2.1 compute_invoice_total_by_day

```
function compute_invoice_total_by_day()
```

Computes the total invoiced amount per day, for closed invoices. Useful for displaying the evolution of total income over time.

Returns a Chart.js-compatible array with labels (dates) and a single dataset (total per day).

6.2.2 compute_invoice_avg_by_day

```
function compute_invoice_avg_by_day()
```

Computes the average invoice amount per day, considering only closed invoices. This helps analyze how invoice sizes vary over time.

Returns a Chart.js-compatible array with labels (dates) and a single dataset (average amount).

6.2.3 compute_invoice_avg_days_to_pay

```
function compute_invoice_avg_days_to_pay()
```

Calculates the average number of days it takes to get paid, based on the difference between invoice and payment dates, grouped by payment day.

Returns a Chart.js-compatible array: labels (payment dates) and one dataset (average days to pay).

6.2.4 compute_top5_customers_by_total

```
function compute_top5_customers_by_total()
```

Retrieves the top 5 customers based on total invoiced amount for closed invoices. Cuts customer names using ‘intelligence_cut()‘ for chart readability.

Returns a Chart.js-compatible array: labels (customer names) and one dataset (total invoiced).

6.2.5 compute_invoice_paid_vs_pending

```
function compute_invoice_paid_vs_pending()
```

Counts how many closed invoices have been paid or are still unpaid. Produces a pie chart data structure separating "Paid" and "Unpaid".

Returns a Chart.js-compatible array: labels (Paid/Unpaid) and one dataset (count of invoices).

6.2.6 compute_invoice_open_vs_closed

```
function compute_invoice_open_vs_closed()
```

Counts how many invoices are currently open or closed. Produces a pie chart data structure separating "Open" and "Closed".

Returns an ECharts-compatible array with one pie series.

6.3 Reverses the process of ‘make_matrix_data()‘ for the ‘invoices‘ app.

```
apps/sales/php/invoices.php
```

This function takes a previously generated matrix-style array of invoice data (grouped as lines, taxes, and totals), compares it with the current values in the database, and builds a minimal diff-like array that includes only the fields that have changed, been removed, or added.

6.3.1 unmake_matrix_data

```
function unmake_matrix_data($json, $invoice_id)
```

Reverse the matrix-encoded invoice data to a diff-style data array.

- @json => The matrix-encoded invoice data (from make_matrix_data)
- @invoice_id => The invoice ID used to retrieve the original data

Return a diff array representing only the differences between the provided data and the current state of the database.

6.3.2 Get next code

```
function get_next_code($table, $field, $prefix)
```

This function returns the next code of a serie, the codes are composed by a prefix that contains the year and a correlative number, the number is computed using the numeric value of the codes that begin by the prefix, by this reason the serie is restarted when the year changes.

- @table => the table that contains the codes
- @field => the field that contains the codes
- @prefix => the prefix of the serie, for example PF2026-

6.3.3 Generates or completes proforma and final invoice metadata.

```
function set_proforma_invoice($json, $invoice_id)
```

This function ensures that the given invoice JSON structure has appropriate values for:

- ‘proforma_code‘ and ‘proforma_date‘ (if not provided or if it’s a new invoice)
- ‘invoice_code‘ and ‘invoice_date‘ (if the invoice is being closed)
- ‘is_closed‘ (based on database if not explicitly set)
- ‘paid_date‘ (if the invoice is marked as paid)
- ‘due_date‘ (if the invoice is marked as closed)

Rules:

- If no ‘invoice_id‘ is given or ‘proforma_code‘ is not defined, a new proforma number is generated.
- If the invoice is being closed (‘is_closed‘ is true), a new invoice number is generated.
- If ‘is_closed‘ is not defined in the JSON, it’s looked up in the database for the given invoice ID.
- If ‘is_paid‘ is true and no ‘paid_date‘ is set, the current date is assigned.
- If the invoice is closed and no ‘due_date‘ is set, the current date is assigned.
- @json => Invoice data to process and complete.
- @invoice_id => ID of the invoice, used to retrieve current status from DB.

Return the updated JSON with appropriate codes and dates filled in.

6.4 Taxes and withholdings helpers

```
apps/sales/php/taxes.php
```

This file contains the functions shared by the documents that use taxes and withholdings in their lines, like the invoices and the quotes, the taxes and the withholdings are stored in the same master table (app_taxes) and are differentiated by the type field (0 tax, 1 withholding), and the company defines how the user operates with each one of them using the tax_mode and withholding_mode fields (0 not used, 1 one by document, 2 by line).

All functions that request a type as argument expect tax or withholding, that are the names used as prefix by the fields of each type.

6.4.1 Get taxes modes

```
function get_taxes_modes()
```

This function returns an array with the mode used by each type, the keys of the array are tax and withholding and the values can be as follow:

- 0 => the type is not used, nothing is requested to the user
- 1 => the user selects only one value for the entire document
- 2 => the user selects one value for each line of the document

6.4.2 Get taxes master

```
function get_taxes_master($type = '')
```

This function returns the active registers of the master table, each row contains the id, name, value, type and default fields.

- @type => the type that you want to retrieve, all types by default

6.4.3 Get taxes column

```
function get_taxes_column($type, $title)
```

This function returns the definition of the column used by the lines of the documents, the column is a dropdown with the values of the requested type when the mode is by line, otherwise the column is hidden, this allow to use the same structure of data for all modes.

- @type => the type of the column, can be tax or withholding
- @title => the title of the column

6.4.4 Get taxes total column

```
function get_taxes_total_column($type, $title)
```

This function returns the definition of the column used by the totals of the documents, the column is hidden when the type is not used.

- @type => the type of the column, can be tax or withholding
- @title => the title of the column

6.4.5 Get taxes subtotal column

```
function get_taxes_subtotal_column($title)
```

This function returns the definition of the subtotal column used by the totals of the documents, the column is hidden when the taxes and the withholdings are not used, because the subtotal is the same that the total.

- @title => the title of the column

6.4.6 Get taxes rows

```
function get_taxes_rows($type)
```

This function returns the rows used by the select of the one by document mode, the first row is void to allow documents without tax or withholding.

- @type => the type of the rows, can be tax or withholding

6.4.7 Get taxes default

```
function get_taxes_default($type)
```

This function returns the value of the default register of the requested type, or a void string if there is not a default register.

- @type => the type of the default, can be tax or withholding

6.4.8 Make taxes data

```
function make_taxes_data($lines)
```

This function returns the data of the lines used by the documents, the argument is the list of the lines with the description, quantity, price, discount, tax_id, tax_value, withholding_id, withholding_value and total fields, and the returned array contains the matrix of the lines with the description, quantity, price, discount, tax, withholding and total, and the values used by the selects of the one by document mode, that only are set when all lines of the document are using the same value.

- @lines => the list of the lines of the document

7 Tester

7.1 Rating widget example

```
apps/tester/js/rating.js
```

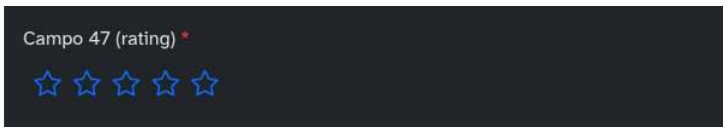
This file is an example of a widget provided by an application, each widget is registered in saltos.bootstrap.__field using the name of the type, and since then, it can be used in the layouts as any other widget of SaltOS, for example, <rating id="xxx"/>.

The data input fields must store in element.saltos the object returned by the helper saltos.bootstrap.__saltos_helper, and replace the features that need a different behavior, in this case, the framework (get_data, check_required, form_disabled, form.data and reset) only uses these features and does not need to know anything about the new widgets.

7.1.1 Rating constructor helper

```
saltos.bootstrap.__field.rating = field
```





This function returns an object of the rating type, it is a data input field that shows five stars and stores the selected value (from 1 to 5) in a hidden input.

- @id => the id used by the object
- @class => allow to add more classes to the stars container
- @value => the value used as value parameter (from 1 to 5, void or 0 for no stars)
- @disabled => this parameter raise the disabled flag
- @required => this parameter raise the required flag
- @tooltip => this parameter raise the title flag
- @label => this parameter is used as text for the label
- @color => the color of the stars (primary by default)

Notes:

This widget programs all features of the data input fields: set_value paints the stars, set_disabled disables the buttons of the stars, get_data normalizes the values as integers and check_required colorizes the stars container and focuses the first star.

7.2 Tester application

```
apps/tester/js/tester.js
```

This application implements the typical features associated with a tester, showcasing functionalities such as modals, toasts, and offcanvas elements.

7.2.1 Tester object

```
saltos.tester = {};
```

This object stores all the functions used by the Tester application, providing the necessary methods for interaction and testing features.

7.2.2 Campo 8: Alert functionality

```
saltos.tester.campo8 = ()
```

This function triggers a simple alert box when called, showcasing a button-click behavior.

7.2.3 Campo 9: Modal functionality

```
saltos.tester.campo9 = ()
```

This function displays a Bootstrap modal with a title, body, and footer. The modal includes:

- Text placeholders for content.
- A dropdown menu for additional options.
- Footer buttons to accept or cancel actions, with corresponding console logs.

7.2.4 Campo 10: Offcanvas functionality

```
saltos.tester.campo10 = ()
```

This function displays a Bootstrap offcanvas element with a title and body content. The body includes:

- Text placeholders for content.
- A dropdown menu for additional options.

7.2.5 Campo 11: Toast functionality

```
saltos.tester.campo11 = ()
```

This function displays a Bootstrap toast notification with customizable attributes, including:

- A title and subtitle for context.
- Body content with dynamic timestamp information.

7.2.6 Campo 22: Open statistics page

```
saltos.tester.campo23 = ()
```

This function opens the SaltOS statistics page in a new browser tab or window.

8 Users

8.1 Login application

```
apps/users/js/login.js
```

This application implements the tipical features associated to login

8.1.1 Main object

```
saltos.login = {};
```

This object contains all SaltOS code

8.1.2 Authenticate login function

```
saltos.login.authenticate = async ()
```

This function tries to authenticate the user using the user and pass fields of the form, to do it uses the authenticate function that send data to the authtoken action

If the response of the authtoken action contains the expired flag, this function switches the screen to the renew box instead of showing the usual access denied message, allowing the user to set a new password without needing a valid session

8.1.3 Renew function

```
saltos.login.renew = async ()
```

This function is intended to be used by the renew box shown after an expired login attempt, it collects the new password and its confirmation and, together with the user and the expired password (still held by the hidden user/pass fields of the login box), tries to renew the password using the authrenew action, that returns a valid token on success

Note: the success message uses saltos.app.toast instead of saltos.app.modal, because it is immediately followed by the saltos.app.login event, which fires a hashchange, and every hashchange unconditionally closes any open modal (see hash.js) - a modal here would close itself before ever being seen. The error case above has no screen change after it, so a modal is fine there

8.2 Days functions

```
apps/users/php/days.php
```

This file contain all functions needed by the days feature

8.2.1 Days to bin

```
function days2bin($days)
```

This function tries to convert the days format used by the multiselect to the string expected by the database formed by ones and zeroes to represent if a day is operative for the user or not, for example, the selection 64,32,16,8,4 is returned like from monday to friday (1111100)

- @days => the string containing the days in power of two separated by comma

8.2.2 Bin to days

```
function bin2days($days)
```

This function tries to do the reverse action that the previous function, is able to get an string like 1111100 and returns the list of all bits in decimal like 64,32,16,8,4.

- @days => the string containing the days in binary format

8.2.3 Fix for days

```
function fix4days($data)
```

This function is intended to be used as wrapper in the result of the query that contains an element called days, in the database the days is stored using the binary notation like 1111100, and for the user interface, is needed to translate this string into a decimal string like 64,32,16,8,4.

- @data => the data obtained from an execute_query, for example, they must contain an entry called days.

8.3 Groups functions

```
apps/users/php/groups.php
```

This file contain all functions needed by the groups app

8.3.1 Insert group action

```
function insert_group($data)
```

This action allow to insert registers in the database associated to the groups app and only requires the data.

- @data => the array with all data used to create the new group

8.3.2 Update group action

```
function update_group($group_id, $data)
```

This action allow to update registers in the database associated to the groups app and requires the group_id and data.

- @group_id => the group_id desired to be updated
- @data => the array with all data used to update the group

8.3.3 Delete group action

```
function delete_group($group_id)
```

This action allow to delete registers in the database associated to the groups app and only requires the group_id.

- @group_id => the group_id desired to be deleted

8.4 Matrix functions

```
apps/users/php/matrix.php
```

This file contains all the functions required by the Excel widget for handling permissions and applications in a matrix structure.

8.4.1 Create matrix data

```
function make_matrix_data($perms, $apps, $main, $user)
```

This function generates a matrix of permissions and applications, associating each application with its respective permissions and their assigned values ("Allow", "Deny", or "").

8.4.2 Create matrix cells

```
function make_matrix_cells($perms, $apps, $main, $user)
```

This function generates matrix cells with additional attributes such as column and row indices, and dropdown options for editing cell values. Read-only attributes are applied to specific cells.

8.4.3 Unmake matrix data

```
function unmake_matrix_data($perms, $apps, $main, $json)
```

This function reverses the matrix data into a more manageable format by applying permission and application mappings while validating data consistency.

8.4.4 Generate matrix permissions

```
function make_matrix_perms($table, $field, $id)
```

This function generates the complete matrix data and metadata for use in the Excel widget, including column headers, row headers, data values, and cell details.

8.4.5 Get permissions from database

```
function _get_db_perms()
```

This function returns the active permissions (id and code) from tbl_perms, caching the result in a static variable to avoid repeated queries.

8.4.6 Get applications from database

```
function _get_db_apps()
```

This function returns the active applications (id and code) from tbl_apps, caching the result in a static variable to avoid repeated queries.

8.4.7 Create matrix columns

```
function make_matrix_columns()
```

This function builds the column definitions for the Excel widget, one dropdown column ("Allow"/"Deny") per permission returned by _get_db_perms.

8.4.8 Create matrix rows

```
function make_matrix_rows()
```

This function builds the row definitions for the Excel widget, one row per application returned by _get_db_apps.

8.5 Users functions

```
apps/users/php/users.php
```

This file contain all functions needed by the users app

8.5.1 Insert user action

```
function insert_user($data)
```

This action allow to insert registers in the database associated to the users app and only requires the data.

- @data => the array with all data used to create the new user

8.5.2 Update user action

```
function update_user($user_id, $data)
```

This action allow to update registers in the database associated to the users app and requires the user_id and data.

- @user_id => the user_id desired to be updated
- @data => the array with all data used to update the user

8.5.3 Delete user action

```
function delete_user($user_id)
```

This action allow to delete registers in the database associated to the users app and only requires the user_id.

- @group_id => the user_id desired to be deleted